

Technical Architecture Document (TAD)

BudgetMind: AI Cashflow Forecaster & Crisis Advisor

Version: 1.1

Status: Publish

Lead Architect: Isaac Furqan

Date: Apr 25, 2026

1. Introduction

This document details the underlying technical specifications, infrastructure, and component decomposition for BudgetMind. It focuses on the specific technologies, libraries, and deployment methodologies used to deliver the predictive financial health platform.

2. Technical Stack and Core Libraries

BudgetMind is built entirely on the Python ecosystem, leveraging specialized libraries for data manipulation, visualization, and generative AI integration.

- **Frontend/UI:** Streamlit (v1.x) — Used for rapid development of the interactive, web-based Predictive Dashboard. Manages user input (CSV upload, NLP queries) and presents results.
- **AI Engine:** Google Gemini 1.5 Pro (via Google Generative AI SDK) — The core intelligence layer for the AI Crisis Advisor and Decision Simulator. It handles complex NLP tasks, pattern analysis, and generating human-readable financial advice.
- **Data Processing:** Pandas & NumPy — Utilized for all time-series logic, aggregation, data cleaning, running-balance calculation, and scenario modification in the Decision Simulator.

- **Visualization:** Plotly / Altair — Used by `visualizer.py` to create dynamic, interactive time-series charts for the 90-day cash flow trajectory.

3. Component Decomposition and Responsibilities

The architecture is divided into three primary functional Python modules, ensuring clear separation of concerns (Modular Design).

3.1. `forecaster.py` (Core Logic)

- **Responsibility:** Data ingestion, schema validation, time-series aggregation, and all financial mathematics.
- **Key Functions:**
 - `calculate_running_balance()`: Computes the daily projected liquidity.
 - `identify_shortfalls()`: Flags dates where the balance drops below the "Safe Operating Threshold" or zero.
 - `apply_simulation()`: Integrates 'What-If' scenarios (e.g., new expense lines) back into the `forecast_df`.

3.2. `gemini_client.py` (AI Interface)

- **Responsibility:** Securely manage the connection to the Gemini API and structure prompts for effective inference.
- **Dependencies:** Google Generative AI SDK.
- **Key Functions:**
 - `generate_crisis_advice()`: Sends the "Crisis Context" payload (shortfall dates, expense summaries) to Gemini and returns the natural language mitigation strategy.
 - `parse_what_if_query()`: Translates user's natural language query (e.g., "What if I hire a developer for \$4000/month starting June?") into structured data (cost, date, type) for `forecaster.py` to process.

3.3. visualizer.py (Presentation Utility)

- **Responsibility:** Rendering the interactive charts based on the current `forecast_df` stored in Streamlit's session state.
- **Key Functions:**
 - `render_cashflow_trajectory()`: Generates the main 90-day line chart.
 - `render_threshold_warnings()`: Highlights threshold breach points on the chart.

4. Deployment and Infrastructure

4.1. Runtime Environment

The application is designed to be fully containerized.

- **Packaging:** Docker container for consistent environment and dependencies.
- **Operating System:** Linux-based container (e.g., Alpine or Debian).
- **Language:** Python 3.x.

4.2. Security and Compliance

- **API Key Management:** Gemini 1.5 Pro API keys are strictly managed via environment variables and are never hard-coded or exposed on the client-side.
- **Data Handling:** Transaction data is processed exclusively **in-memory** and is discarded upon session termination, ensuring strict data privacy and compliance.

5. State Management Flow (Streamlit)

The application relies on Streamlit's `st.session_state` to maintain the integrity of the predictive model across user interactions, particularly during simulations.

1. **Initial Load:** CSV ingested, `raw_data` and `forecast_df` populated.
2. **User Input:** User enters a 'What-If' query (NLP text-driven interface).

3. 3. Simulation Path:

- Query is sent to `gemini_client.py`'s `parse_what_if_query`.
- Structured parameters are returned and stored in `simulation_params`.
- `forecaster.py` uses `simulation_params` to create a new, modified `forecast_df`.
- `forecast_df` is updated, triggering `visualizer.py` to re-render the chart in real-time.

This flow ensures the core data remains consistent while allowing rapid, iterative simulation analysis.